

Prompt:

8/17/2026, 9:51:37 PM

lanjut Tahap I

Response:

8/17/2026, 9:51:38 PM

Baik. Kita masuk ke **Tahap I — Backtesting, Walk-Forward Validation, dan Statistical Evaluation**.

Menurut saya, ini justru tahap yang akan menentukan apakah seluruh arsitektur A–H kita benar-benar ilmiah atau hanya terlihat bagus di atas kertas. Kita sudah membangun forecasting, factor model, covariance, regime, QUBO, QAOA, execution, dan position management. Sekarang semuanya harus diuji **tanpa memberikan sistem informasi yang pada saat itu belum tersedia**.

Saya akan membangun Tahap I dari prinsip paling dasar sampai akhirnya menjadi protokol eksperimen yang bisa langsung kita implementasikan.

Tahap I — Backtesting dan Statistical Validation

Tujuan Tahap I:

Historical Data → Causal Simulation → Trading Decisions → PnL → Statistical Inference

Tetapi ada satu prinsip yang harus menjadi fondasi:

Pada waktu t , sistem hanya boleh mengetahui F_t .

Tidak lebih.

I.1. Mengapa backtesting biasa berbahaya?

Bayangkan kita mempunyai data:

$$P_1, P_2, \dots, P_T.$$

Kemudian kita melatih model menggunakan seluruh data:

$$P_{1:T}.$$

Setelah itu kita menguji strategy pada data yang sama.

Misalnya diperoleh:

$$Sharpe = 3.7.$$

Ini hampir tidak berarti.

Sebab model sudah melihat masa depan.

Secara matematis kita ingin:

$$\theta_t = \text{Train}(D_{\leq t})$$

bukan:

$$\theta = \text{Train}(D_{\leq T}).$$

I.2. Causal information set

Pada waktu:

$$t,$$

definisikan:

$$F_t = \sigma(D_1, D_2, \dots, D_t).$$

Artinya semua keputusan harus merupakan fungsi:

$$a_t = \pi(F_t).$$

Bukan:

$$a_t = \pi(F_{t+1}).$$

Ini kelihatannya sederhana, tetapi hampir seluruh sumber **look-ahead bias** berasal dari pelanggaran konsep ini.

I.3. Contoh look-ahead bias

Misalkan kita menggunakan:

$$r_t = \ln \frac{P_t}{P_{t-1}}.$$

Jika pada akhir candle t kita membuat keputusan untuk:

$$t + 1,$$

maka menggunakan:

$$r_t$$

masih sah.

Tetapi jika kita menggunakan:

$$r_{t+1},$$

maka kita sudah menggunakan informasi masa depan.

I.4. Bahkan preprocessing bisa menyebabkan leakage

Misalnya kita melakukan standardization:

$$Z_t = \frac{x_t - \mu}{\sigma}.$$

Jika:

$$\mu = \frac{1}{T} \sum_{t=1}^T x_t,$$

maka μ menggunakan data masa depan.

Seharusnya:

$$\mu_t = \frac{1}{N_t} \sum_{\tau \leq t} x_\tau.$$

Demikian juga:

$$\sigma_t.$$

I.5. Jadi seluruh pipeline harus causal

Bukan hanya model forecasting.

Harus:

$$\begin{aligned} Feature_t &= f(D_{\leq t}) \\ Scaler_t &= f(D_{\leq t}) \\ Forecast_t &= f(Feature_t, \theta_t) \\ Covariance_t &= f(D_{\leq t}) \\ QUBO_t &= f(\mu_t, \Sigma_t, \dots) \\ QAOA_t &= f(QUBO_t) \\ Order_t &= f(QAOA_t). \end{aligned}$$

I.6. Train/validation/test

Kita tidak menggunakan random train-test split.

Untuk time series:

Time must flow forward.

Misalnya data:

2021 – 2025.

Kita bisa membaginya:

$$\underbrace{2021 - 2023}_{Train} \rightarrow \underbrace{2024}_{Validation} \rightarrow \underbrace{2025}_{Test}.$$

Tetapi untuk sistem kita saya ingin sesuatu yang lebih kuat:

Walk-Forward Validation.

I.7. Walk-forward validation

Misalkan:

$$T_0$$

adalah awal training.

Kita mempunyai:

$$Train_1 = [t_1, t_2]$$

dan:

$$Test_1 = [t_3, t_4].$$

Kemudian:

$$Train_2 = [t_1, t_4]$$

dan:

$$Test_2 = [t_5, t_6].$$

Begitu seterusnya.

Secara umum:

$$\theta_k = Train(D_{\leq T_k})$$

kemudian:

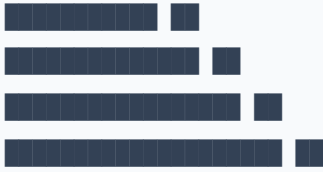
$$Performance_k = Test(\theta_k, D_{T_k+1:T_{k+1}}).$$

I.8. Expanding window

Contohnya:

TRAIN

TEST



Training semakin panjang.

Secara matematis:

$$T_k = [t_0, T_k].$$

Ini cocok jika informasi historis lama masih relevan.

I.9. Rolling window

Alternatif:

TRAIN

TEST



Training window tetap:

$$W.$$

Sehingga:

$$T_k = [T_k - W, T_k].$$

Ini lebih cocok jika market mengalami structural change.

I.10. Untuk sistem kita: hybrid

Saya menyarankan:

rolling estimation + expanding model validation.

Misalnya:

- covariance: rolling 60–120 hari,
- beta: rolling 60–120 hari,
- regime: rolling,
- forecasting model: retrain berkala,
- hyperparameter validation: expanding walk-forward.

Ini lebih masuk akal secara finansial.

I.11. Nested walk-forward

Kalau kita ingin benar-benar research-grade, bahkan pemilihan hyperparameter tidak boleh menggunakan test period.

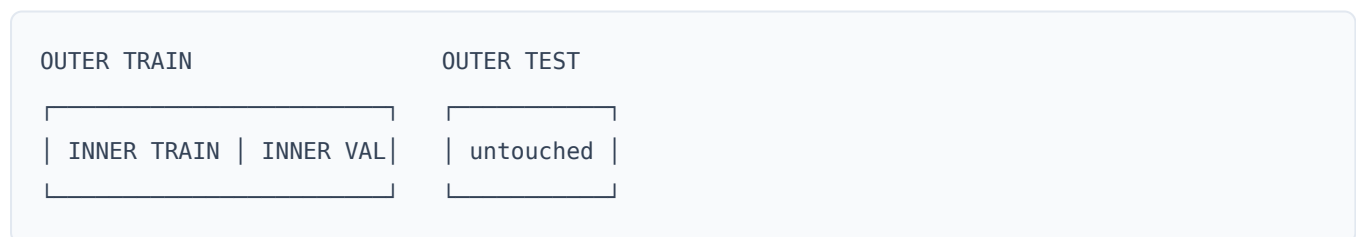
Strukturnya:

Outer Test

dan di dalam training period:

Inner Train → Inner Validation.

Jadi:



Ini mencegah kita memilih model berdasarkan test performance.

I.12. Contoh konkret

Misalnya:

2021 – 01 – 01 → 2025 – 12 – 31.

Outer test:

2025.

Training:

2021 – 2024.

Di dalam 2021–2024:

- 2021–2023 → inner train,
- 2024 → inner validation.

Pilih:

θ^*

berdasarkan validation.

Kemudian **bekukan model**.

Baru jalankan:

2025

sebagai out-of-sample test.

I.13. Tetapi ada masalah yang lebih besar

Kita bukan hanya mempunyai satu model.

Kita mungkin mencoba:

- 5 feature sets,
- 4 forecasting models,
- 3 covariance estimators,
- 5 QAOA depths,
- 4 risk coefficients,
- 3 transaction cost assumptions.

Jumlah kombinasi:

$$5 \times 4 \times 3 \times 5 \times 4 \times 3 = 3600.$$

Kalau kita memilih yang menghasilkan Sharpe tertinggi, kita telah melakukan:

multiple testing.

I.14. Backtest overfitting

Misalkan semua strategi sebenarnya tidak punya alpha:

$$E[SR] = 0.$$

Tetapi kita mencoba 1.000 strategi.

Kemungkinan besar ada beberapa yang memperoleh:

$$SR > 2.$$

Bukan karena benar-benar bagus.

Tetapi karena:

selection by luck.

I.15. Inilah mengapa kita perlu experiment registry

Setiap eksperimen harus memiliki:

ExperimentID

dan mencatat:

FeatureSet
Model
Hyperparameters
CovarianceModel
QUBOP arameters
QAOAP arameters
TransactionCost
DateRange

sehingga kita tahu berapa banyak eksperimen yang sebenarnya sudah kita lakukan.

I.16. Kita harus membedakan tiga jenis hasil

In-sample

$IS.$

Out-of-sample

$OOS.$

Paper/live

$LIVE.$

Targetnya:

$$Performance_{IS} \geq Performance_{OOS} \approx Performance_{LIVE}.$$

Tidak harus sama, tetapi divergence harus masuk akal.

I.17. Return portfolio

Misalkan keputusan portfolio:

$w_t.$

Return:

$$R_{p,t+1} = w_t^T r_{t+1}.$$

Tetapi ini belum memasukkan transaction cost.

I.18. Transaction cost

Misalkan:

$$\Delta w_t = w_t - w_{t-1}.$$

Cost sederhana:

$$C_t = c^T |\Delta w_t|.$$

Maka:

$$R_{net,t+1} = w_t^T r_{t+1} - c^T |\Delta w_t|.$$

Ini minimal harus ada.

I.19. Spread dan slippage

Untuk trading jangka pendek, kita sebaiknya menggunakan:

$$C_t = C_t^{commission} + C_t^{spread} + C_t^{slippage}.$$

Misalnya:

$$C_t = \kappa_t^T |\Delta q_t|.$$

Dengan:

$$\kappa_t = \text{effective cost per unit.}$$

I.20. Market impact

Jika order besar:

$$Q_t$$

dibandingkan liquidity:

$$ADV_t,$$

cost meningkat.

Model sederhana:

$$Impact_t = \eta \left(\frac{|Q_t|}{ADV_t} \right)^\gamma.$$

Misalnya:

$$\gamma \approx \frac{1}{2}.$$

Sehingga:

$$Impact \propto \sqrt{\frac{|Q|}{ADV}}.$$

Ini jauh lebih realistis daripada fixed transaction cost.

I.21. Net PnL

Maka:

$$PnL_t = w_t^T r_{t+1} - TC_t - Impact_t.$$

Semua metrik performance harus menggunakan:

$$PnL_{net}.$$

Bukan gross PnL.

I.22. Equity curve

Mulai dari:

$$V_0.$$

Kemudian:

$$V_t = V_{t-1}(1 + R_{net,t}).$$

Atau log wealth:

$$\ln V_t = \ln V_{t-1} + \ln(1 + R_{net,t}).$$

I.23. Cumulative return

$$R_{cum} = \frac{V_T}{V_0} - 1.$$

Tetapi ini **tidak cukup**.

Strategi:

+100%

dengan:

$MDD = -80\%$

sangat berbeda dari:

+80%

dengan:

$MDD = -10\%$.

I.24. Maximum Drawdown

Peak:

$$M_t = \max_{\tau \leq t} V_\tau.$$

Drawdown:

$$DD_t = \frac{V_t - M_t}{M_t}.$$

Maximum drawdown:

$$MDD = \min_t DD_t.$$

I.25. Sharpe ratio

Jika:

$$\bar{R} = \frac{1}{T} \sum_t R_t,$$

dan:

$$s_R = Std(R_t),$$

maka:

$$SR = \frac{\bar{R} - r_f}{s_R} \sqrt{A}.$$

A adalah annualization factor.

Misalnya hourly:

$$A \approx 252 \times 24$$

untuk pasar yang berjalan 24/7, meskipun annualization harus disesuaikan dengan definisi return dan jam trading yang digunakan.

I.26. Tetapi Sharpe memiliki kelemahan

Sharpe menganggap:

$$\sigma$$

sebagai risk measure yang cukup.

Padahal downside:

$$R_t < 0$$

lebih penting bagi trader.

Maka:

$$\text{Sortino} = \frac{\bar{R} - r_f}{\sigma_{\text{downside}}} \sqrt{A}.$$

I.27. Calmar ratio

Untuk sistem trading yang mengalami drawdown:

$$\text{Calmar} = \frac{\text{CAGR}}{[\text{MDD}]}$$

Ini sangat berguna untuk membandingkan strategy dengan risk profile berbeda.

I.28. Tail risk

Kita juga perlu:

$$VaR_{\alpha}.$$

Misalnya:

$$VaR_{95\%}.$$

Tetapi saya lebih tertarik pada:

$$CVaR_{\alpha}.$$

Expected Shortfall:

$$CVaR_{\alpha} = E[-L \mid L \geq VaR_{\alpha}].$$

Ini menjawab:

Jika kita masuk ke 5% kondisi terburuk, rata-rata kerugiannya berapa?

I.29. Trading-specific metrics

Kita juga perlu:

$$HitRate = \frac{\#profitabletrades}{\#trades}.$$

Tetapi hit rate bisa menipu.

Misalnya:

$$HitRate = 80\%$$

namun satu loss besar menghapus seluruh profit.

Maka kita membutuhkan:

$$ProfitFactor = \frac{\sum Profit}{|\sum Loss|}.$$

I.30. Expectancy

Untuk setiap trade:

$$W = \text{average win}$$

$$L = \text{average loss}$$

$$p = P(\text{win}).$$

Expected value:

$$E = pW - (1 - p)L.$$

Strategy profitable jika:

$$E > 0.$$

I.31. Tetapi untuk sistem kita, expectancy harus conditional

Kita memiliki signal strength:

$$s_t.$$

Maka kita bisa menghitung:

$$E[PnL \mid s_t \in bin_k].$$

Misalnya:

Signal	Expected PnL
weak	-0.01%
medium	+0.03%
strong	+0.12%
very strong	+0.18%

Jika hubungan seperti ini konsisten OOS, signal kita lebih meyakinkan.

I.32. Information Coefficient

Untuk cross-sectional trading, kita dapat menghitung:

$$IC_t = \text{Corr}(\mu_t, r_{t+\Delta}).$$

Kemudian:

$$IC_{avg} = \frac{1}{T} \sum_t IC_t.$$

Ini mengukur apakah model dapat melakukan ranking asset.

I.33. Rank IC

Karena portfolio optimization sering hanya membutuhkan ranking:

$$RankIC_t = Corr_{Spearman}(Rank(\hat{\mu}_t), Rank(r_{t+\Delta})).$$

Ini sering lebih robust terhadap outlier.

I.34. Forecast quality $\neq$ portfolio quality

Kita harus melakukan dua evaluasi terpisah.

Forecast layer

$$\hat{\mu} \rightarrow IC.$$

Portfolio layer

$$\hat{\mu}, \Sigma \rightarrow Portfolio \rightarrow PnL.$$

Ini sangat penting ketika nanti membandingkan QAOA.

I.35. Eksperimen utama kita

Kita ingin membandingkan:

QAOA vs Classical Solver

tetapi **dengan problem yang identik**.

Misalnya:

$$\min_x x^T Q_t x.$$

Jangan membandingkan QAOA dengan classical optimizer pada objective berbeda.

I.36. Classical baseline

Kita bisa menggunakan:

Exact brute force

untuk:

$$N \leq 20$$

misalnya.

Karena jumlah konfigurasi:

$$2^N.$$

Ini memungkinkan kita mengetahui:

$$x_{exact}^*$$

I.37. Mengukur approximation ratio

Misalkan objective minimization:

$$C(x).$$

QAOA menghasilkan:

$$C_{QAOA}.$$

Exact:

$$C^*.$$

Maka:

$$\boxed{Gap = \frac{C_{QAOA} - C^*}{|C^*|}.$$

Semakin kecil:

$$Gap \rightarrow 0,$$

semakin dekat dengan optimum.

I.38. Probability of finding optimum

QAOA tidak selalu menghasilkan optimum.

Misalnya:

$$P(x^*) = 0.37.$$

Artinya jika kita melakukan sampling:

$$37\%$$

probabilitas menghasilkan konfigurasi optimal.

Kita dapat mencatat:

$$P_{opt.}$$

I.39. Expected objective dari QAOA

Jika output distribution:

$$p_{\theta}(x),$$

maka:

$$E[C] = \sum_x p_{\theta}(x) C(x).$$

Ini bahkan lebih informatif daripada hanya best sample.

I.40. Quantum advantage belum tentu PnL advantage

Ini sangat penting untuk skripsi/proyek kita.

Misalnya:

$$Gap_{QAOA} < Gap_{classical}.$$

Tetapi setelah trading:

$$PnL_{QAOA} < PnL_{classical}.$$

Mengapa?

Karena:

estimation error

dan:

transaction cost

mungkin mendominasi optimization quality.

I.41. Jadi kita harus punya tiga level benchmark

Level 1 — Mathematical

QAOA vs *Exact*.

Mengukur:

objective gap.

Level 2 — Computational

QAOA vs *Classical optimizer*.

Mengukur:

- runtime,
 - iterations,
 - objective,
 - scalability.
-

Level 3 — Financial

$$Portfolio_{QAOA} \text{ vs } Portfolio_{classical}$$

Mengukur:

- PnL,
- Sharpe,
- MDD,
- turnover,
- CVaR,
- transaction cost.

I.42. Ini menghasilkan eksperimen yang sangat kuat

Kita bisa mempunyai:

<i>Method</i>	<i>Performance</i>
<i>Exact</i>	...
<i>Simulated Annealing</i>	...
<i>MILP/QP</i>	...
<i>Classical heuristic</i>	...
<i>QAOA</i>	...

Kemudian:

same data + same QUBO + same constraints + same costs.

Barulah perbandingan adil.

I.43. Kita juga perlu baseline trading

Bukan hanya classical optimizer.

Bandingkan terhadap:

Buy & Hold

B&H.

Equal Weight

$$w_i = \frac{1}{N}.$$

Momentum

$$w_i \propto M_i.$$

Mean-Variance

$$\max_w \mu^T w - \lambda w^T \Sigma w.$$

Classical combinatorial optimizer

$$QUBO_{\text{classical}}.$$

QAOA

$$QUBO_{\text{QAOA}}.$$

I.44. Dengan begitu kita dapat menjawab pertanyaan yang jauh lebih menarik

Bukan:

"Apakah QAOA menghasilkan profit?"

Tetapi:

Apakah QAOA menghasilkan solusi yang lebih baik daripada classical optimization **untuk masalah combinatorial portfolio/trading yang sama?**

Dan:

Apakah improvement tersebut cukup besar untuk bertahan setelah estimation error, transaction cost, slippage, dan regime changes?

I.45. Stress testing

Backtest normal belum cukup.

Kita perlu stress:

Market Regimes.

Misalnya:

Bull

$$r_{market} > 0, \quad \sigma \text{ low.}$$

Bear

$$r_{market} < 0.$$

Crisis

$$\sigma \gg \text{median.}$$

Sideways

$$|\mu| \approx 0.$$

Liquidity shock

$$\text{spread} \uparrow, \quad \text{depth} \downarrow .$$

I.46. Factor stress

Karena sistem kita menggunakan:

DXY, Gold, SPX, BTC, ETH,

kita dapat melakukan scenario:

$$DXY + 2\sigma$$

atau:

$$SPX - 3\sigma.$$

Kemudian hitung:

$$\Delta P_{portfolio}.$$

I.47. Stress test portfolio

Misalnya portfolio memiliki factor exposure:

$$E = B^T w.$$

Shock:

$$\Delta f.$$

Approximate PnL:

$$\Delta P \approx E^T \Delta f.$$

Ini sangat powerful.

Kita dapat mengetahui:

"Jika USD tiba-tiba menguat 2σ , apa yang terjadi pada portfolio?"

I.48. Scenario-based stress

Secara umum:

$$f^{stress} = f_t + \Delta f.$$

Kemudian:

$$\mu^{stress} = \mu(f^{stress})$$

dan:

$$\Sigma^{stress} = \Sigma(f^{stress}).$$

Lalu kita dapat membangun:

$$Q^{stress}.$$

Bahkan:

$$QAOA(Q^{stress}).$$

I.49. Ini berarti QAOA dapat menjadi regime-adaptive

Normal:

$$Q_t^{normal}.$$

Stress:

$$Q_t^{stress}.$$

Crisis:

$$Q_t^{crisis}.$$

Dengan demikian sistem tidak sekadar:

buy/sell.

Tetapi:

re-optimize portfolio under changing market state.

I.50. Monte Carlo

Kita juga perlu menguji apakah performance hanya terjadi karena satu trajectory historis.

Misalkan residual:

$$\epsilon_t.$$

Kita dapat menghasilkan synthetic paths:

$$\epsilon_t^{(b)}.$$

untuk:

$$b = 1, \dots, B.$$

Kemudian menjalankan strategy pada:

$$B$$

simulasi.

I.51. Bootstrap

Kita dapat menggunakan block bootstrap karena return time-series tidak IID sempurna.

Ambil block:

$$(r_t, \dots, r_{t+k})$$

kemudian resample block.

Ini mempertahankan sebagian temporal dependence.

I.52. Distribusi Sharpe

Misalnya dari:

$$B = 1000$$

simulasi kita memperoleh:

$$SR^{(1)}, SR^{(2)}, \dots, SR^{(B)}.$$

Kemudian kita bisa melihat:

$$P(SR > 0).$$

Bahkan:

5%, 50%, 95%

quantile.

I.53. Probabilistic performance

Daripada mengatakan:

Sharpe = 1.72

kita bisa mengatakan:

$$SR_{median} = 1.72$$

dengan:

$$95\% CI = [0.83, 2.41].$$

Ini jauh lebih ilmiah.

I.54. Deflated Sharpe Ratio

Karena kita melakukan banyak eksperimen:

$$SR_{observed}$$

harus dikoreksi terhadap multiple testing.

Konsep yang relevan adalah:

$$Deflated\ Sharpe\ Ratio.$$

Intinya:

semakin banyak strategi/model yang kita coba, semakin tinggi Sharpe yang dapat muncul secara kebetulan.

I.55. Probability of Backtest Overfitting

Kita juga dapat menggunakan pendekatan:

$$PBO$$

untuk mengukur probabilitas strategy yang terlihat terbaik di-sample akan gagal out-of-sample.

Ini akan sangat berguna ketika kita mulai bereksperimen dengan:

- feature,
- QUBO coefficient,
- QAOA depth,
- risk penalty,
- stop loss,
- take profit.

I.56. Parameter QAOA juga bisa overfit

Misalnya kita memilih:

$$p = 1, 2, 3, \dots, 10$$

dan:

$$\beta, \gamma$$

berdasarkan profit OOS.

Hati-hati.

Kita telah melakukan hyperparameter optimization.

Jadi:

$$p$$

harus dipilih berdasarkan training/validation.

Test tidak boleh disentuh.

I.57. Quantum hyperparameter validation

Misalnya:

$$p \in \{1, 2, 3\}$$

dan optimizer:

$$\{SPSA, Adam\}.$$

Kita pilih berdasarkan:

$$V_{\text{validation Objective}}.$$

Kemudian:

$$T_{\text{test}}$$

hanya digunakan sekali.

I.58. Tetapi ada pertanyaan yang lebih penting

Apakah kita benar-benar membutuhkan QAOA?

Kita harus memberikan kesempatan kepada classical method untuk menang.

Ini penting secara ilmiah.

Jika classical optimizer ternyata:

$$10\times$$

lebih cepat dan menghasilkan:

$$\text{identical portfolio},$$

maka itu adalah hasil yang valid.

Kita tidak boleh memaksakan:

$$QAOA > \textit{Classical}.$$

I.59. Justru hasil negatif dapat menjadi penelitian yang bagus

Misalnya hasil:

$$\textit{Objective}_{QAOA} = \textit{Objective}_{\textit{classical}} + 0.5\%.$$

Tetapi:

$$\textit{Runtime}_{QAOA} > \textit{Runtime}_{\textit{classical}}.$$

Maka kesimpulan:

Pada ukuran masalah dan simulator/hardware yang digunakan, QAOA belum memberikan computational advantage.

Itu jauh lebih kredibel daripada memaksakan klaim quantum advantage.

I.60. Eksperimen ablation

Kita juga perlu menghapus komponen satu per satu.

Misalnya:

$$\textit{Model}_A$$

tanpa factor.

$$\textit{Model}_B$$

dengan factor.

$$\textit{Model}_C$$

dengan factor + regime.

$$\textit{Model}_D$$

dengan factor + regime + QAOA.

Kemudian:

$$Performance_A \rightarrow Performance_B \rightarrow Performance_C \rightarrow Performance_D.$$

I.61. Contoh ablation table

Model	Factor	Regime	QAOA	Sharpe
Baseline	×	×	×	0.72
Factor	✓	×	×	0.91
Factor + Regime	✓	✓	×	1.08
Factor + Regime + Classical	✓	✓	Classical	1.21
Factor + Regime + QAOA	✓	✓	QAOA	1.24

Jika hasil seperti ini terjadi secara OOS, kita memiliki cerita penelitian yang sangat kuat.

I.62. Kita juga perlu turnover analysis

Turnover:

$$TO_t = \frac{1}{2} \sum_i |w_{i,t} - w_{i,t-1}|.$$

Kenapa faktor:

$$\frac{1}{2}?$$

Karena setiap pembelian biasanya mempunyai sisi penjualan counterpart dalam rebalancing portfolio.

I.63. Alpha vs turnover

Kita ingin mengetahui:

$$PnL_{gross}$$



terhadap:

$$TO.$$

Misalnya:

$$PnL_{gross} = 10\%$$

tetapi:

$$TC = 12\%.$$

Strategy sebenarnya tidak viable.

Maka kita membutuhkan:

$$\boxed{Net\ Alpha = Gross\ Alpha - Transaction\ Cost.}$$

I.64. Capacity

Untuk hedge fund/prop firm, profit tidak cukup.

Kita juga bertanya:

$$\boxed{\text{Berapa besar capital yang dapat strategy tanggung?}}$$

Jika:

$$AUM$$

naik, market impact:

$$Impact \uparrow .$$

Maka:

$$PnL_{net}(AUM)$$

akhirnya menurun.

I.65. Capacity curve

Kita dapat menghasilkan:

$$AUM \rightarrow Net\ Sharpe.$$

Misalnya:

AUM	$Sharpe$
\$10k	2.1
\$100k	1.9
\$1M	1.4
\$10M	0.6

Ini jauh lebih dekat dengan quantitative trading profesional.

I.66. Execution realism

Untuk setiap signal:

$$t.$$

Kita tidak boleh mengasumsikan:

$$Fill_t = 100\%.$$

Harus ada:

$$P(fill_t \mid order, market).$$

Misalnya limit order:

$$P(fill) = f(distance\ from\ mid, queue, volume, volatility).$$

I.67. Limit order

Jika kita memasang:

$$P_{limit} = P_{mid} - \delta$$

untuk long.

Maka fill hanya terjadi jika market menyentuh atau menembus level tersebut **dan** order kita mendapatkan execution.

Jadi backtest idealnya memiliki:

FillModel.

I.68. Stop-loss juga harus realistis

Misalnya stop:

$$P_{SL}.$$

Jika market gap:

$$P_t < P_{SL},$$

kita tidak boleh menganggap fill tepat:

$$P_{SL}.$$

Harus menggunakan:

$$P_{fill}$$

yang mungkin lebih buruk.

I.69. Ini sangat penting untuk crypto

Karena:

- volatility tinggi,
- spread berubah,
- liquidity berubah,
- liquidation cascade,
- market order impact.

Jadi:

bukan guaranteed price.

I.70. Evaluation hierarchy

Saya ingin sistem kita melewati:

Level 0 : Unit Tests

kemudian:

Level 1 : Mathematical Validation

kemudian:

Level 2 : Historical Backtest

kemudian:

Level 3 : Walk – Forward

kemudian:

Level 4 : Stress Test

kemudian:

Level 5 : Paper Trading

kemudian:

Level 6 : Small Live Capital

I.71. Unit test matematis

Contohnya kita harus memastikan:

$$\Sigma = \Sigma^T.$$

dan:

$$\lambda_{\min}(\Sigma) \geq 0.$$

Jika tidak:

$$\Sigma$$

bukan positive semidefinite.

Kemudian:

$$Q$$

harus memiliki bentuk yang benar.

I.72. Test QUBO

Untuk:

$$N$$

kecil, lakukan brute force:

$$x \in \{0, 1\}^N.$$

Hitung:

$$C(x)$$

untuk semua:

$$2^N$$

kombinasi.

Kemudian:

$$x^* = \arg \min_x C(x).$$

Bandingkan dengan QAOA.

I.73. Test portfolio

Pastikan:

$$\sum_i w_i = 1$$

jika fully invested.

Pastikan:

$$w_i \geq 0$$

untuk long-only.

Jika cardinality:

$$\sum_i x_i \leq K.$$

Harus selalu dipenuhi.

I.74. Test causality

Ini salah satu unit test paling penting.

Ambil data sampai:

$$t.$$

Jalankan model:

$$Prediction_t.$$

Kemudian tambahkan data:

$$t + 1.$$

Prediction pada waktu t **tidak boleh berubah**.

Jika berubah:

look-ahead leakage.

I.75. Ini dapat kita jadikan automated test

Secara konsep:

$$Model(D_{\leq t}) = Model(D_{\leq t+1})|_{prediction_t}$$

harus menghasilkan:

identical.

Ini salah satu test yang sangat berharga untuk pipeline kita.

I.76. Test survivorship bias

Universe asset juga tidak boleh hanya berisi aset yang masih hidup sekarang.

Misalnya:

2021

ada 100 saham.

Pada:

2026

hanya 70 yang masih ada.

Kalau backtest 2021 menggunakan hanya 70 saham yang masih hidup, kita sudah menggunakan future information.

I.77. Untuk crypto

Masalahnya sedikit berbeda karena banyak token:

- listing,
- delisting,
- liquidity collapse.

Universe harus berdasarkan:

$$Universe_t.$$

bukan:

$$Universe_{2026}.$$

I.78. Corporate action

Untuk saham internasional kita harus memperhatikan:

- stock split,
- dividend,
- rights issue,
- merger,
- delisting.

Return yang digunakan harus konsisten.

I.79. Currency normalization

Ini penting karena kita ingin:

$$\text{crypto} + \text{US equities} + \text{international equities} + \text{Gold} + \text{USD}.$$

Portfolio base currency misalnya:

$$USD.$$

Jika asset dalam:

$$EUR, JPY, GBP,$$

return harus memperhitungkan FX.

Jika:

$$P_{asset}^{EUR}$$

dan:

$$FX_{USD/EUR},$$

maka USD price:

$$P^{USD} = P^{EUR} \times FX_{USD/EUR}.$$

I.80. Ini berarti FX juga menjadi factor

Untuk international equities:

$$r_{USD} = r_{local} + r_{FX} + r_{local}r_{FX}.$$

Dengan log return:

$$r^{USD} = r^{local} + r^{FX}.$$

Ini salah satu alasan mengapa:

$$USD$$

menjadi market factor penting.

I.81. Financial objective akhir

Kita akhirnya dapat mendefinisikan:

$$J = E[R_{net}] - \lambda Var(R_{net}) - \eta CVaR - \kappa Turnover$$

dan QUBO kita merupakan pendekatan diskrit terhadap objective ini.

I.82. Tetapi evaluasi juga menggunakan objective tersebut

Jangan hanya:

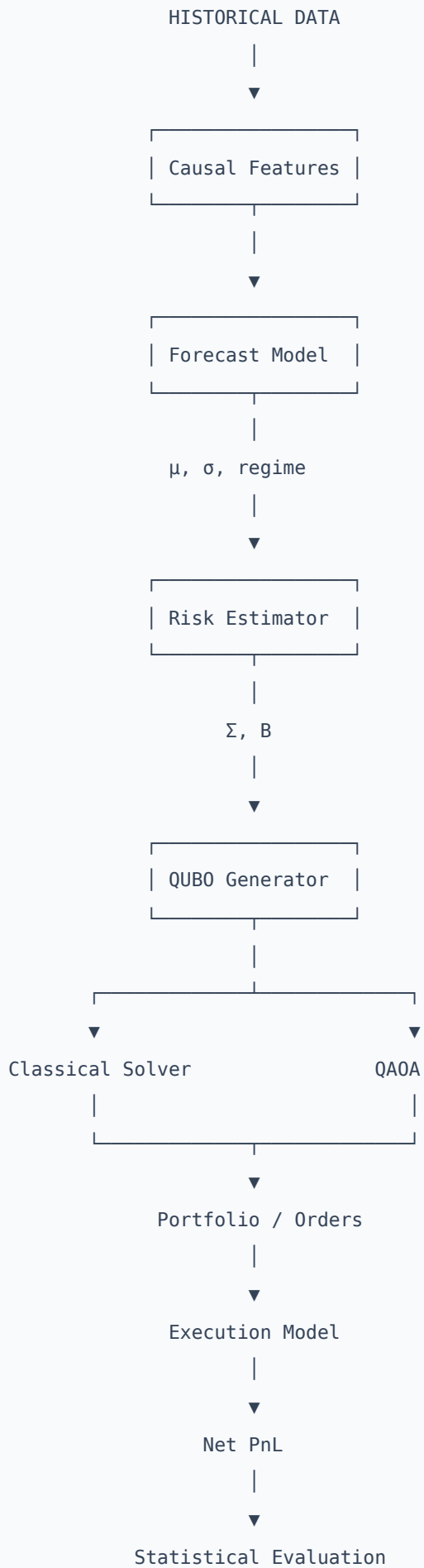
$$PnL.$$

Kita evaluasi:

<i>Return</i>
<i>Sharpe</i>
<i>Sortino</i>
<i>Calmar</i>
<i>MDD</i>
<i>CV aR</i>
<i>Turnover</i>
<i>Transaction Cost</i>
<i>Capacity</i>

I.83. Framework eksperimen final

Saya menyarankan eksperimen utama kita nantinya seperti ini:



I.84. Dan eksperimen kita harus memiliki "frozen test set"

Ini saya anggap **aturan keras**.

Misalnya:

2021 – 2024

digunakan untuk seluruh pengembangan.

Kemudian:

2025

adalah:

LOCKED TEST SET.

Kita tidak melihat performance 2025 untuk memilih:

- feature,
- model,
- QAOA depth,
- λ ,
- stop loss,
- take profit,
- rebalance interval.

Setelah semua desain selesai:

ONE FINAL RUN ON 2025.

I.85. Bahkan lebih baik: temporal holdout

Kalau data memungkinkan:

2021 – 2023 → *development*

2024 → *validation*

2025 → *test*

2026 → *paper/liveforward*.

Dengan tanggal sekarang **17 Agustus 2026**, data sampai periode sekarang nantinya dapat diperlakukan sebagai forward period setelah seluruh parameter dikunci.

I.86. Apa yang sebenarnya kita ingin buktikan?

Setelah Tahap I, penelitian kita seharusnya bisa menjawab empat pertanyaan:

Pertanyaan 1

Apakah forecasting model memiliki predictive power?

$$IC > 0?$$

Pertanyaan 2

Apakah portfolio optimization meningkatkan risk-adjusted return?

$$SR_{\text{optimized}} > SR_{\text{baseline}}?$$

Pertanyaan 3

Apakah QAOA mampu menemukan solusi QUBO yang kompetitif?

$$Gap_{\text{QAOA}} \approx Gap_{\text{classical/exact}}?$$

Pertanyaan 4

Apakah keunggulan tersebut bertahan setelah:

$cost + slippage + market\ impact + parameter\ uncertainty + regime\ changes?$

Pertanyaan keempat adalah yang paling penting.

I.87. Dan ada satu eksperimen yang menurut saya wajib kita lakukan

Kita buat **empat strategy**:

$$S_1 = \text{Equal Weight}$$

$$S_2 = \text{Forecast} + \text{Classical Optimization}$$

$$S_3 = \text{Forecast} + \text{QAOA}$$

$$S_4 = \text{Forecast} + \text{QAOA} + \text{Dynamic Position Management.}$$

Kemudian:

$$S_1 \rightarrow S_2$$

mengukur manfaat optimization.

$$S_2 \rightarrow S_3$$

mengukur manfaat QAOA.

$$S_3 \rightarrow S_4$$

mengukur manfaat dynamic trade management.

Ini merupakan **ablation study** yang sangat bersih.

I.88. Bahkan kita dapat melakukan "QAOA replacement test"

Ambil QUBO:

$$Q_t.$$

Jangan ubah apa pun.

Jalankan:

$$x_t^{\text{QAOA}}$$

dan:

$$x_t^{classical}.$$

Kemudian:

$$PnL(QAOA)$$

vs.

$$PnL(classical).$$

Dengan demikian semua perbedaan berasal dari **optimizer**, bukan dari data atau forecasting.

Ini adalah eksperimen yang menurut saya paling penting jika tujuan akhirnya adalah penelitian quantum optimization.

I.89. Struktur final penelitian kita

Dengan Tahap I, keseluruhan framework A–I sekarang menjadi:

<i>A</i>	Market Problem
<i>B</i>	Market State
<i>C</i>	Alpha/Forecasting
<i>D</i>	Risk
<i>E</i>	Position Management
<i>F</i>	Execution
<i>G</i>	QUBO + Ising + QAOA
<i>H</i>	State Estimation
<i>I</i>	Validation + Backtesting

Dan semuanya terhubung dalam satu persamaan konseptual:

$$D_{\leq t} \xrightarrow{E} S_t \xrightarrow{F} (\mu_t, \Sigma_t, B_t, \tau_t, R_t) \xrightarrow{Q} Q_t \xrightarrow{QAOA} x_t^* \xrightarrow{\text{Sizing}} w_t^* \xrightarrow{\text{Execution}} PnL_{t+1}.$$

Kemudian:

$$PnL_{t+1} \rightarrow S_{t+1},$$

sehingga sistem menjadi **closed-loop dynamic system**.

Hal yang menurut saya paling penting setelah Tahap I

Kita **sudah tidak lagi sekadar mendesain strategi trading**. Kita sekarang memiliki kerangka eksperimen yang memungkinkan kita memisahkan tiga sumber keberhasilan:

Prediction

dari:

Optimization

dari:

Execution.

Ini sangat penting untuk proyekmu. Kalau nanti QAOA memberikan hasil lebih baik, kita dapat mengatakan *di mana* peningkatannya terjadi. Kalau tidak lebih baik, kita juga dapat menunjukkan apakah masalahnya berasal dari QAOA, forecasting, covariance estimation, atau execution.

Dan menurut saya, **setelah Tahap I, tahap berikutnya sebaiknya bukan langsung "Tahap J = coding"**. Kita perlu terlebih dahulu membuat **Tahap J — Formal Mathematical Specification**, yaitu menyatukan A–I menjadi satu sistem matematis lengkap: mendefinisikan semua variabel, indeks waktu, asset universe, state vector, forecast, covariance, factor exposure, objective, constraint, QUBO, Ising Hamiltonian, QAOA circuit, position sizing, TP/SL, dan transition equation secara konsisten. Setelah itu barulah kita punya *blueprint* yang cukup ketat untuk diterjemahkan ke kode tanpa ada bagian algoritma yang ambigu.